

TP 0 — Découvrir Docker Compose

Conteneurs et DevOps — Exercice d'introduction, non noté

ESGI — Youssef Younes

2026

Table des matières

À lire avant tout	1
Image et conteneur : les deux mots à connaître	1
Étape 1 — Un conteneur à la main	2
Étape 2 — La même chose, écrite dans un fichier	2
Étape 3 — Un volume : votre fichier dans le conteneur	3
Étape 4 — Deux services qui se parlent	3
Étape 5 — La mémoire qui disparaît, puis qui reste	4
Étape 6 — Changer la configuration sans toucher au fichier	5
Ce que vous avez appris	6
Vérifiez que vous avez compris	6
Annexe — Réponses	6
Annexe — Les commandes du TP	7
Annexe — Si ça ne marche pas	8

À lire avant tout

Ce TP n'est pas noté. Il sert à une seule chose : que Docker Compose ne soit plus une boîte noire quand vous attaquerez le TP StayBook. On part d'un conteneur lancé à la main, et on arrive, en six petites étapes, à deux programmes qui se parlent, gardent leurs données et se configurent sans toucher au code.

Tout le code est fourni. Vous ne modifiez que des fichiers `docker-compose.yml`, et toujours de quelques lignes. Prenez le temps de lire les explications entre les commandes : c'est là que se trouve ce qu'il faut retenir.

Image et conteneur : les deux mots à connaître

Une **image** est un paquet figé qui contient un programme et tout ce dont il a besoin pour tourner : le système de fichiers, les bibliothèques, la configuration par défaut. On la télécharge depuis un dépôt public, Docker Hub, ou on la construit soi-même. Une image ne fait rien : elle attend, comme un programme installé mais pas lancé.

Un **conteneur** est une image en train de tourner. On peut lancer dix conteneurs à partir de la même image. Un conteneur peut être arrêté, supprimé, recréé : tout ce qu'il a écrit à l'intérieur disparaît avec lui.

Retenez l'analogie : l'image est le moule, le conteneur est le gâteau.

Décompressez l'archive fournie par l'enseignant. Le dossier TP0 contient un dossier par étape. Ouvrez un terminal dedans.

Étape 1 — Un conteneur à la main

Pas encore de Compose. On lance un serveur web nginx avec la commande de base de Docker :

```
docker run -d -p 8080:80 --name mon-nginx nginx:alpine
```

Lisez la commande de droite à gauche :

- `nginx:alpine` : l'**image** à utiliser. `nginx` est le programme, `alpine` la variante légère. Docker la télécharge si besoin.
- `--name mon-nginx` : un nom pour retrouver le conteneur.
- `-p 8080:80` : le port **8080 de votre machine** est relié au port **80 du conteneur**. Sans cette option, le serveur tourne mais personne ne peut l'atteindre.
- `-d` : en arrière-plan, pour récupérer la main dans le terminal.

Ouvrez `http://localhost:8080`. La page d'accueil de nginx s'affiche. Puis :

```
docker ps                # les conteneurs qui tournent : le vôtre est là
docker logs mon-nginx    # ce que le programme a écrit : vous y voyez votre visite
docker stop mon-nginx    # arrêter
docker rm mon-nginx      # supprimer
docker ps                # plus rien
```

Ça fonctionne, mais imaginez neuf programmes, chacun avec sa longue commande, à lancer dans le bon ordre, tous les matins. C'est exactement le problème que Docker Compose résout.

Étape 2 — La même chose, écrite dans un fichier

Entrez dans le dossier `etape2`. Il contient un seul fichier, `docker-compose.yml` :

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
```

Comparez avec la commande de l'étape 1 : c'est **la même information**, rangée dans un fichier.

- `services:` : la liste des conteneurs à lancer. Ici un seul.
- `web:` : le **nom** que l'on donne à ce conteneur. On le choisit librement.
- `image:` : remplace le `nginx:alpine` de la commande.
- `ports:` : remplace le `-p 8080:80`. Les guillemets sont obligatoires ici, car YAML se méfie des deux-points.

Le fichier est en **YAML** : les espaces en début de ligne ont un sens. `web` est indenté sous `services`, `image` est indenté sous `web`. Toujours **2 espaces**, jamais de tabulation.

```
docker compose up -d      # lit le fichier et lance tout ce qu'il décrit
docker compose ps         # l'équivalent de docker ps, limité à ce fichier
docker compose logs web   # les logs du service « web »
```

Rechargez `http://localhost:8080`. Même page qu'à l'étape 1. Puis :

```
docker compose down # arrête ET supprime tout ce que le fichier avait créé
```

Retenez les trois commandes : `up` -d, `ps`, `down`. Vous les utiliserez cent fois.

Étape 3 — Un volume : votre fichier dans le conteneur

Dossier `etape3`. Il contient le même fichier avec deux lignes de plus, et un dossier `site` avec une page HTML :

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./site:/usr/share/nginx/html:ro
```

La ligne `volumes` se lit : le dossier `./site` **de ma machine** apparaît dans le conteneur à l'emplacement `/usr/share/nginx/html`, en lecture seule (`ro`). C'est le dossier où `nginx` cherche les pages à servir.

```
docker compose up -d
```

Ouvrez `http://localhost:8080` : ce n'est plus la page par défaut, c'est la vôtre. Maintenant, ouvrez `site/index.html` dans un éditeur, changez le titre, enregistrez, rechargez le navigateur. Le changement est immédiat, sans redémarrer : le conteneur lit directement votre dossier.

```
docker compose down
```

Un **volume** est un pont entre l'intérieur du conteneur et l'extérieur. Ici il sert à donner un fichier au conteneur. À l'étape 5, il servira à l'inverse : garder ce que le conteneur écrit.

Étape 4 — Deux services qui se parlent

Dossier `etape4`. Cette fois, une vraie application : un **compteur de visites**. Elle est composée de deux programmes :

- `web` : une petite application Python qui affiche une page et compte les visites ;
- `redis` : une base de données en mémoire, où le compteur est stocké.

```
navigateur → web (Python, port 5000) → redis (port 6379)
port 8080
```

Le fichier :

```
services:
  web:
    build: ./app
    ports:
      - "8080:5000"
    environment:
      REDIS_HOST: redis
      MESSAGE: Bonjour
    depends_on:
      - redis
```

```
redis:
  image: redis:7-alpine
```

Trois nouveautés.

build: `./app` au lieu de `image:`. Il n'existe pas d'image toute faite de notre compteur : Docker la **construit** à partir du dossier `./app`, qui contient le code Python et un fichier `Dockerfile` (la recette de construction). Vous n'avez pas à lire ces fichiers, mais ouvrez-les si vous êtes curieux : ils sont courts.

environment: : des variables passées au programme au démarrage. `REDIS_HOST: redis` dit à l'application « la base de données s'appelle `redis` ». Et `redis`, c'est le **nom du deuxième service**, trois lignes plus bas. Compose crée un réseau privé pour ce fichier, et sur ce réseau, chaque service est joignable par son nom. Pas d'adresse IP à connaître, pas de configuration réseau : le nom suffit.

depends_on: : démarrer `redis` avant `web`, puisque `web` en a besoin.

Remarquez aussi ce que `redis` **n'a pas** : pas de ports. Seul `web` a besoin de lui parler, et `web` est sur le même réseau privé. Depuis votre machine, Redis est invisible. On n'ouvre que ce qui doit l'être.

```
docker compose up -d --build      # --build : construire l'image de web avant de lancer
docker compose ps                  # deux conteneurs
```

Ouvrez `http://localhost:8080`. Rechargez plusieurs fois : le compteur monte. Chaque rechargement, c'est `web` qui demande à `redis` d'ajouter 1, en l'appelant par son nom.

Pour voir la traduction du nom en adresse, demandez-la depuis l'intérieur du conteneur `web` :

```
docker compose exec web getent hosts redis
```

Une adresse IP s'affiche à côté de `redis`. C'est Docker qui l'a attribuée et qui tient l'annuaire.

Étape 5 — La mémoire qui disparaît, puis qui reste

Toujours dans `etape4`. Notez la valeur du compteur, puis :

```
docker compose down
docker compose up -d
```

Rechargez la page : le compteur est **revenu à 1**. `down` a supprimé le conteneur `redis`, et tout ce qu'il avait en mémoire est parti avec lui. En production, ce serait la perte de toutes les données.

La solution est un **volume nommé**. Ajoutez au service `redis` :

```
redis:
  image: redis:7-alpine
  volumes:
    - redis-data:/data
```

et, tout en bas du fichier, au même niveau que `services:`, déclarez ce volume :

```
volumes:
  redis-data:
```

Le dossier /data du conteneur, là où Redis enregistre ses données, est maintenant stocké dans un espace géré par Docker, nommé `redis-data`, qui **n'appartient pas au conteneur**. Relancez, rechargez plusieurs fois, puis :

```
docker compose up -d          # applique la modification
# rechargez la page plusieurs fois
docker compose down
docker compose up -d
```

Le compteur a **gardé sa valeur**. Le conteneur a été détruit et recréé, mais pas le volume.

```
docker volume ls              # le volume est là, il s'appelle etape4_redis-data
```

Deux sortes de volumes, donc : à l'étape 3, un dossier de votre machine (`./site`), pratique pour donner des fichiers ; ici, un volume nommé, fait pour conserver ce que le conteneur écrit. Le seul moyen de le supprimer est de le demander explicitement : `docker compose down -v`.

Étape 6 — Changer la configuration sans toucher au fichier

Le message « Bonjour » est écrit dans le YAML. Pour le changer, il faut éditer le fichier. Ce n'est pas grave ici, mais imaginez un mot de passe : on ne veut pas qu'il soit dans un fichier partagé avec toute l'équipe.

Remplacez la ligne du message par :

```
MESSAGE: ${MESSAGE}
```

et créez, à côté du `docker-compose.yml`, un fichier nommé `.env` contenant une ligne :

```
MESSAGE=Bienvenue dans Docker Compose
```

```
docker compose config | grep MESSAGE    # affiche la valeur remplacée : Compose a
lu .env
docker compose up -d
```

Rechargez : le titre de la page a changé. Changez la valeur dans `.env`, relancez `up -d`, rechargez : il change encore. Le fichier YAML, lui, n'a pas bougé.

C'est la règle en production : le YAML décrit la **structure**, le fichier `.env` porte les **valeurs qui changent** d'une machine à l'autre ou qui ne doivent pas être partagées. On versionne le premier, jamais le second.

Terminez proprement :

```
docker compose down -v
```

Ce que vous avez appris

Mot-clé	Découvert à l'étape	Il répond à la question
services	2	Quels conteneurs je lance ?
image	2	Quel programme tout fait j'utilise ?
ports	2	Quel port j'ouvre vers ma machine ?
volumes (dossier local)	3	Quels fichiers je donne au conteneur ?
build	4	Où est le code à construire ?
environment	4	Quelle configuration je lui passe ?
depends_on	4	Qui doit démarrer avant ?
volumes (nommé)	5	Où conserver ce que le conteneur écrit ?
\${VARIABLE} et .env	6	Comment changer une valeur sans toucher au YAML ?

Et trois idées qui valent plus que les mots-clés :

1. **Les services se parlent par leur nom.** `REDIS_HOST: redis` suffit, Docker tient l'annuaire.
2. **Un conteneur est jetable, ses données ne doivent pas l'être.** D'où les volumes nommés.
3. **On n'ouvre vers l'extérieur que ce qui doit l'être.** Un seul `ports:`, sur le service que le navigateur appelle.

Dans le TP StayBook, vous retrouverez exactement ces neuf lignes de tableau, avec neuf conteneurs au lieu de deux. Il n'y aura rien de nouveau, seulement plus de services.

Vérifiez que vous avez compris

Répondez pour vous, puis comparez avec les réponses en annexe.

1. Quelle est la différence entre une image et un conteneur ?
2. Dans `"8080:5000"`, quel nombre désigne votre machine, et lequel le conteneur ?
3. Pourquoi le service `redis` n'a-t-il pas de ligne `ports:` ? Serait-ce une erreur d'en ajouter une ?
4. Dans `REDIS_HOST: redis`, qui fait le lien entre le mot `redis` et une adresse réseau ?
5. Après `docker compose down`, qu'est-ce qui est supprimé et qu'est-ce qui est conservé ? Et avec `down -v` ?
6. Pourquoi mettre `MESSAGE` dans `.env` plutôt que directement dans le YAML ?

Annexe — Réponses

1. L'image est le paquet figé (le moule), le conteneur est l'image en cours d'exécution (le gâteau). Plusieurs conteneurs peuvent tourner à partir d'une même image.

2. Le premier, 8080, est le port de votre machine. Le second, 5000, est celui du programme dans le conteneur.
3. Seul web lui parle, et ils sont sur le même réseau privé. Ajouter `ports`: ne casserait rien mais exposerait la base de données à tout le monde sur votre machine : inutile et risqué.
4. Docker Compose. Il crée un réseau privé pour le fichier et y tient un annuaire (DNS) où chaque service est inscrit sous son nom.
5. `down` supprime les conteneurs et le réseau, mais garde les volumes nommés et les images. `down -v` supprime aussi les volumes nommés, donc les données.
6. Pour changer la valeur sans modifier le fichier de structure, et pour garder hors du fichier partagé les valeurs sensibles ou propres à une machine. Le YAML se versionne, le `.env` non.

Annexe — Les commandes du TP

Commande	Ce qu'elle fait
<code>docker compose up -d</code>	Lance tout ce que décrit le fichier, en arrière-plan
<code>docker compose up -d --build</code>	Idem, en reconstruisant d'abord les images <code>build</code> :
<code>docker compose ps</code>	Liste les conteneurs du fichier et leur état
<code>docker compose logs <service></code>	Affiche ce qu'un service a écrit. Ajoutez <code>-f</code> pour suivre en continu
<code>docker compose exec <service> <commande></code>	Exécute une commande à l'intérieur d'un conteneur
<code>docker compose config</code>	Affiche le fichier tel que Compose le comprend, variables remplacées. Détecte les erreurs
<code>docker compose down</code>	Arrête et supprime conteneurs et réseau. Garde les volumes
<code>docker compose down -v</code>	Idem, et supprime aussi les volumes nommés

Annexe — Si ça ne marche pas

Ce que vous voyez	Ce que ça veut dire	Que faire
<code>port is already allocated</code>	Le port 8080 est déjà utilisé, souvent par un conteneur de l'étape précédente	<code>docker ps</code> puis <code>docker rm -f <nom></code> , ou <code>docker compose down</code> dans l'autre dossier
<code>yaml: ... did not find expected key</code>	Indentation incorrecte	Comparer avec le fichier de l'énoncé, 2 espaces par niveau
<code>no configuration file provided</code>	Vous n'êtes pas dans le bon dossier	<code>cd etapeN</code>
<code>variable is not set</code>	Le fichier <code>.env</code> manque ou la variable n'y est pas	Créer <code>.env</code> à côté du <code>docker-compose.yml</code>
La page ne change pas après modification	Le navigateur a gardé l'ancienne version	Recharger avec <code>Cmd+Shift+R</code> ou <code>Ctrl+Shift+R</code>